

Terraform-05

- 1). Watch the **Terraform-05** video.
- 2). Execute the script shown in the video.

Write script for create an ec2 instance:

```
main.tf x
main.tf > resource "aws_instance" "test-server"

1 resource "aws_instance" "test-server" {
2   ami           = "ami-0e38835daf6b8a2b9"
3   instance_type = "t3.micro"
4   key_name      = "mumbai"
5
6   tags = {
7     Name = "test-server"
8   }
9
10  provisioner "local-exec" {
11    command = "echo instance ${self.public_ip} created! >> instance_ip.txt"
12  }
13 }
```

Terraform init:

```
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...
Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/aws
- Using previously-installed hashicorp/aws v6.50.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
```

Terraform plan:

```
PS C:\Users\DELL\Desktop\Terraform> terraform plan
aws_iam_user.admin_user: Refreshing state... [id=bhargav]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

# aws_iam_user.admin_user will be destroyed
# (because aws_iam_user.admin_user is not in configuration)
- resource "aws_iam_user" "admin_user" {
  - arn           = "arn:aws:iam::922345236105:user/bhargav" -> null
  - force_destroy = false -> null
}
```

terraform apply:

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply
aws_iam_user.admin_user: Refreshing state... [id=bhargav]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

# aws_iam_user.admin_user will be destroyed
# (because aws_iam_user.admin_user is not in configuration)
- resource "aws_iam_user" "admin_user" {
  - arn          = "arn:aws:iam::922345236105:user/bhargav" -> null
  - force_destroy = false -> null
  - id           = "bhargav" -> null
  - name         = "bhargav" -> null
}
```

```
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.test-server: Creating...
aws_iam_user.admin_user: Destroying... [id=bhargav]
aws_iam_user.admin_user: Destruction complete after 2s
aws_instance.test-server: Still creating... [00m10s elapsed]
aws_instance.test-server: Provisioning with 'local-exec'...
aws_instance.test-server (local-exec): Executing: ["cmd" "/c" "echo instance 13.201.42.216 created! >> instance_ip.txt"]
aws_instance.test-server: Creation complete after 16s [id=i-03ece422abc14d832]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\DELL\Desktop\Terraform>
```

verify:

The screenshot shows the AWS Management Console 'Instances' page. At the top, there's a header with 'Instances (1/10) Info', a search bar, and buttons for 'Connect', 'Instance state', 'Actions', and 'Launch instances'. Below the header is a table of instances. The 'test-server' instance is selected and highlighted in blue. Below the table, the details for the 'test-server' instance (ID: i-03ece422abc14d832) are shown. The 'Instance summary' section includes the Instance ID, Public IPv4 address (13.201.42.216), Private IPv4 addresses (172.31.38.242), Instance state (Running), and Public DNS (ec2-13-201-42-216.ap-south-1.compute.amazonaws.com).

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
ubuntu	i-06b487a7d49b8e126	Stopped	t3.micro	-	ap-south-1a	-
Jenkins-Server	i-0d0fe03fd86a34cf6	Stopped	t3.micro	-	ap-south-1a	-
test-server	i-03ece422abc14d832	Running	t3.micro	Initializing	ap-south-1a	ec2-13-201-42-216.ap-...

i-03ece422abc14d832 (test-server)

Instance summary

Instance ID i-03ece422abc14d832	Public IPv4 address 13.201.42.216 open address	Private IPv4 addresses 172.31.38.242
IPv6 address -	Instance state Running	Public DNS ec2-13-201-42-216.ap-south-1.compute.amazonaws.com open address

Execute command:

```
instance_ip.txt
1 instance 13.201.42.216 created!
2
```

```
main.tf > resource "local_sensitive_file" "test"

1 resource "local_sensitive_file" "test" {
2   filename = "pets.txt"
3   content = "i love dogs"
4   provisioner "local-exec" {
5     command = "echo created file pets.txt > file.txt"
6   }
7 }
```

```
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/aws

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.
```

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply
aws_instance.test-server: Refreshing state... [id=i-03ece422abc14d832]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

# aws_instance.test-server will be destroyed
# (because aws_instance.test-server is not in configuration)
- resource "aws_instance" "test-server" {
  - ami = "ami-0e38835daf6b8a2b9" -> null
  - arn = "arn:aws:ec2:ap-south-1:922345236105:instance/i-03ece422abc14d832" -> null
  - associate_public_ip_address = true -> null
  - availability_zone = "ap-south-1a" -> null
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Only 'yes' will be accepted to approve.

Enter a value: yes

local_sensitive_file.test: Creating...
local_sensitive_file.test: Provisioning with 'local-exec'...
local_sensitive_file.test (local-exec): Executing: ["cmd" "/c" "echo created file pets.txt > file.txt"]
local_sensitive_file.test: Creation complete after 0s [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]
aws_instance.test-server: Destroying... [id=i-03ece422abc14d832]
aws_instance.test-server: Still destroying... [id=i-03ece422abc14d832, 00m10s elapsed]
aws_instance.test-server: Still destroying... [id=i-03ece422abc14d832, 00m20s elapsed]
aws_instance.test-server: Still destroying... [id=i-03ece422abc14d832, 00m30s elapsed]
aws_instance.test-server: Destruction complete after 30s

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\DELL\Desktop\Terraform>

```

The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays a file tree for a project named 'TERRAFORM'. The files listed are: `.terraform`, `.terraform.lock.hcl`, `file.txt` (selected), `instance_ip.txt`, `main.tf`, `pets.txt`, `terraform.tfstate`, and `terraform.tfstate.backup`. The main editor window shows the content of `file.txt`, which contains two lines: `1 created file pets.txt` and `2`. A search bar at the top right of the editor shows the text `aws_instance`.

```
main.tf > resource "local_file" "test" > provisioner "local-exec"
```

```
1 resource "local_file" "test" {
2   filename = "pets.txt"
3   content = "i love dogs"
4   provisioner "local-exec" {
5     when = "destroy"
6     command = "echo destroying file pets.txt > file.txt"
7   }
8 }
```

```
PS C:\Users\DELL\Desktop\Terraform> terraform destroy
local_file.test: Refreshing state... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# local_file.test will be destroyed
- resource "local_file" "test" {
  - content      = "i love dogs" -> null
  - content_base64sha256 = "Z4bFH8+Z8MOLj5NceqSTVrrGES69Br2ia8WdkH64V48=" -> null
  - content_base64sha512 = "Z5/x0zCNtkZY7S/4HF-jwHU12yv1GBtOMochV5A9+OVhb/Ps8700fdVbD7pAlJtp/no5E2ICd4l4Yw/HX0ztTmg==" -> null
  - content_md5      = "ef989a06c8df361c85a98135dc8cf42d" -> null
  - content_sha1      = "c524a39c02f142ba0b81da289f2e11332d59b4dd" -> null
  - content_sha256     = "6786c51fcfb3f0c38b8d235c7aa49d356bac6112ebd06bda26bc59d907eb8540f" -> null
}
```

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

local_file.test: Destroying... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]

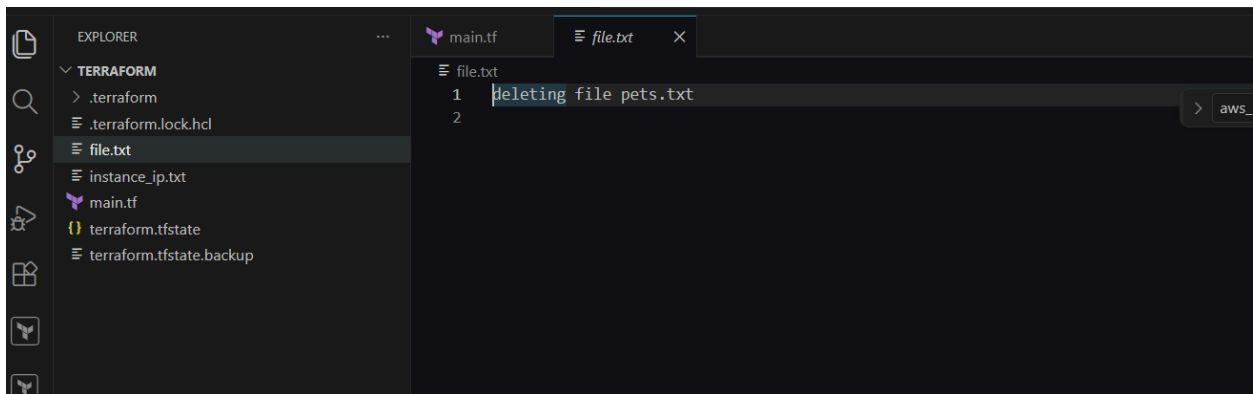
local_file.test: Provisioning with 'local-exec'...

local_file.test (local-exec): Executing: ["cmd" "/C" "echo deleting file pets.txt > file.txt"]

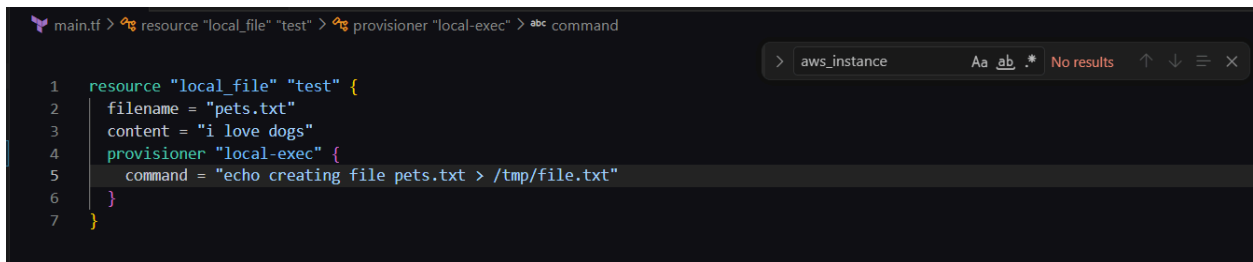
local_file.test: Destruction complete after 0s

Destroy complete! Resources: 1 destroyed.

PS C:\Users\DELL\Desktop\Terraform>



Gave a wrong path



Terraform apply:

If we give any wrong path and do terraform apply then in the state file you can

```
commands will detect it and remind you to do so if necessary).
PS C:\Users\DELL\Desktop\Terraform> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.test will be created
+ resource "local_file" "test" {
  + content           = "i love dogs"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5        = (known after apply)
  + content_sha1        = (known after apply)
}
```

```
local_file.test: Creating...
local_file.test: Provisioning with 'local-exec'...
local_file.test (local-exec): Executing: ["cmd" "/C" "echo creating file pets.txt > /tmp/file.txt"]
local_file.test (local-exec): The system cannot find the path specified.

Error: local-exec provisioner error

with local_file.test,
on main.tf line 4, in resource "local_file" "test":
  4:   provisioner "local-exec" {
    ~~~~~

Error running command 'echo creating file pets.txt > /tmp/file.txt': exit status 1. Output: The system cannot find the path specified.

PS C:\Users\DELL\Desktop\Terraform>
```

```
main.tf x terraform.tfstate
main.tf > resource "local_file" "test" > provisioner "local-exec" > abc command

> aws_instance Aa ab.* No results ↑

1 resource "local_file" "test" {
2   filename = "pets.txt"
3   content = "i love dogs"
4   provisioner "local-exec" {
5     command = "echo creating file pets.txt > file.txt"
6   }
7 }
```

tainted:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\Terraform> terraform taint local

Error: Invalid address

    on line 1:
   (source code not available)

The keyword "local" is reserved and cannot be used to target a resource address. If you are targeting a resource type t
prefix your address with "resource.".

PS C:\Users\DELL\Desktop\Terraform> terraform taint local_file.test
Resource instance local_file.test has been marked as tainted.
PS C:\Users\DELL\Desktop\Terraform> 
```

Status shows tainted

```
main.tf terraform.tfstate x

() terraform.tfstate > ...
6   "outputs": {},
7   "resources": [
8     {
9       "mode": "managed",
10      "type": "local_file",
11      "name": "test",
12      "provider": "provider[\"registry.terraform.io/hashicorp/local\"]",
13      "instances": [
14        {
15          "status": "tainted",
16          "schema_version": 0,
17          "attributes": {
18            "content": "i love dogs",
19            "content_base64": null,
20            "content_base64sha256": "Z4bFH8+z8MOLjSNceqSTVrrGES69Br2ia8wdkH64VA8=",
21            "content_base64sha512": "ZS/x0zcNTkZY7S/4HFjwHU12yv1GBtOMochV5A9+OVHb/Ps8700fdVbd7pAlJtP/no5E2ICD4H4YWHXHztTMg==",
22            "content_md5": "ef909a06c8df361c85a98135dc8cf42d",
23            "content_sha1": "c524a39c02f142ba0b81da289f2e11332d59b4dd",
24            "content_sha256": "6786c51fcfb3f0c38b8d235c7aa49356bac6112ebd06bda26bc59d907eb8540f",
```

Terraform taint local_file.test And terraform apply : replaced

```
PS C:\Users\DELL\Desktop\Terraform> terraform taint local_file.test
Resource instance local_file.test has been marked as tainted.
PS C:\Users\DELL\Desktop\Terraform> terraform apply
local_file.test: Refreshing state... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.test is tainted, so must be replaced
-/+ resource "local_file" "test" {
  ~ content_base64sha256 = "Z4bFH8+z8MOLjSNceqSTVrrGES69Br2ia8wdkH64VA8=" -> (known after apply)
  ~ content_base64sha512 = "ZS/x0zcNTkZY7S/4HFjwHU12yv1GBtOMochV5A9+OVHb/Ps8700fdVbd7pAlJtP/no5E2ICD4H4YWHXHztTMg==" -> (known after apply)
  ~ content_md5 = "ef909a06c8df361c85a98135dc8cf42d" -> (known after apply)
  ~ content_sha1 = "c524a39c02f142ba0b81da289f2e11332d59b4dd" -> (known after apply)
```

Terraform untaint local_file.test Terraform apply

```

PS C:\Users\DELL\Desktop\Terraform> terraform taint local_file.test
Resource instance local_file.test has been marked as tainted.
PS C:\Users\DELL\Desktop\Terraform> terraform untaint local_file.test
Resource instance local_file.test has been successfully untainted.
PS C:\Users\DELL\Desktop\Terraform> terraform apply
local_file.test: Refreshing state... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\DELL\Desktop\Terraform>

```

ERROR:

```

main.tf > resource "local_file" "test" > provisioner "local-exec" > abc command

1 resource "local_file" "test" {
2   filename = "pets.txt"
3   content = "i love dogs"
4   provisioner "local-exec" {
5     command = "echo creating file pets.txt > file.txt"
6   }
7 }

```

set-Item -path env:TF_LOG -value "ERROR"

```

PS C:\Users\DELL\Desktop\Terraform> set-Item -path env:TF_LOG -value "ERROR"
PS C:\Users\DELL\Desktop\Terraform> terraform plan
local_file.test: Refreshing state... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\Users\DELL\Desktop\Terraform>

```

Import:

```

main.tf x terraform.tfstate
main.tf > resource "aws_instance" "manual" > ami

1 resource "aws_instance" "manual" {
2   ami = "ami-0f5ee92e2d63afc18"
3   instance_type = "t3.micro"
4 }
5 }

```

Terraform init:


```

PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.50.0...
- Installed hashicorp/aws v6.50.0 (signed by HashiCorp)

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/local

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

```

Terraform import aws_instance.manual <instance id>

```

PS C:\Users\DELL\Desktop\Terraform> terraform import aws_instance.manual i-0d0fe03fd86a34cf6
aws_instance.manual: Importing from ID "i-0d0fe03fd86a34cf6"...
aws_instance.manual: Import prepared!
  Prepared aws_instance for import
aws_instance.manual: Refreshing state... [id=i-0d0fe03fd86a34cf6]

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.

```

Terraform destroy:

```

PS C:\Users\DELL\Desktop\Terraform> terraform destroy
local_file.test: Refreshing state... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]
aws_instance.manual: Refreshing state... [id=i-0d0fe03fd86a34cf6]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# aws_instance.manual will be destroyed
- resource "aws_instance" "manual" {
  - ami                        = "ami-0f5ee92e2d63afc18" -> null
  - arn                      = "arn:aws:ec2:ap-south-1:922345236105:instance/i-0d0fe03fd86a34cf6" -> null
  - associate_public_ip_address = false -> null
  - availability_zone         = "ap-south-1a" -> null
  - disable_api_stop          = false -> null

```

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```

local_file.test: Destroying... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]
local_file.test: Destruction complete after 0s
aws_instance.manual: Destroying... [id=i-0d0fe03fd86a34cf6]
aws_instance.manual: Still destroying... [id=i-0d0fe03fd86a34cf6, 00m10s elapsed]
aws_instance.manual: Destruction complete after 11s

```

Destroy complete! Resources: 2 destroyed.

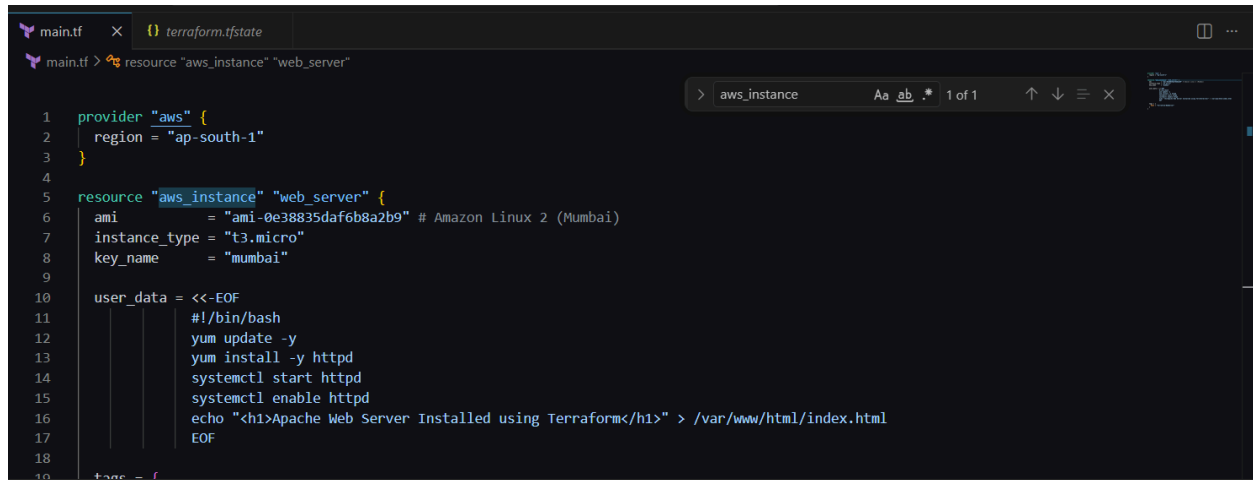
```

PS C:\Users\DELL\Desktop\Terraform> ^C
PS C:\Users\DELL\Desktop\Terraform>

```

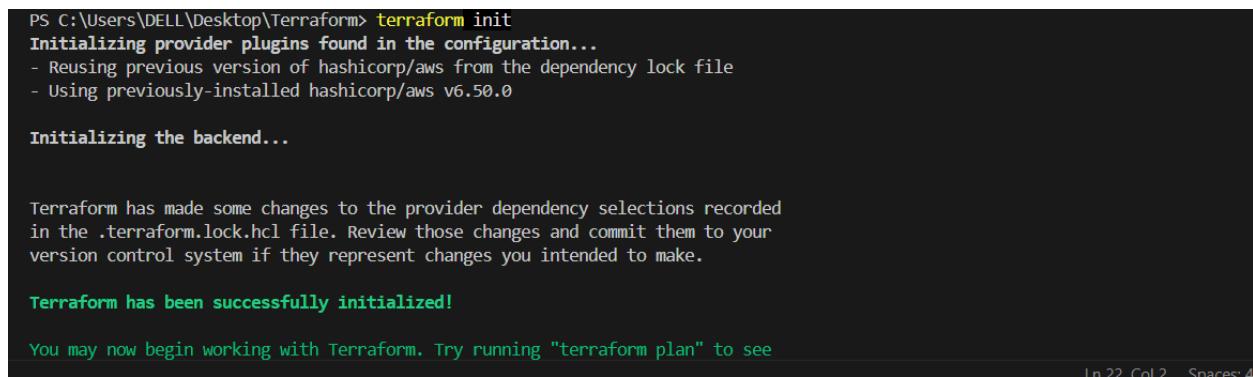
3). Create one **EC2 instance** with **httpd installed** using a Terraform script.

Write a script for create ec2 instance with httpd:



```
1 provider "aws" {
2   region = "ap-south-1"
3 }
4
5 resource "aws_instance" "web_server" {
6   ami           = "ami-0e38835daf6b8a2b9" # Amazon Linux 2 (Mumbai)
7   instance_type = "t3.micro"
8   key_name      = "mumbai"
9
10  user_data = <<-EOF
11    #!/bin/bash
12    yum update -y
13    yum install -y httpd
14    systemctl start httpd
15    systemctl enable httpd
16    echo "<h1>Apache Web Server Installed using Terraform</h1>" > /var/www/html/index.html
17  EOF
18 }
19
```

Terraform init:



```
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

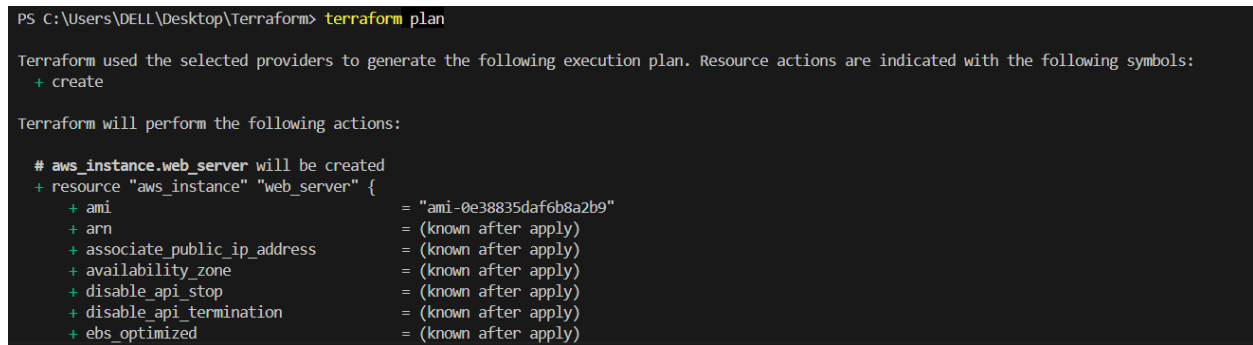
Initializing the backend...

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
```

Terraform plan:



```
PS C:\Users\DELL\Desktop\Terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web_server will be created
+ resource "aws_instance" "web_server" {
  + ami           = "ami-0e38835daf6b8a2b9"
  + arn           = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone           = (known after apply)
  + disable_api_stop           = (known after apply)
  + disable_api_termination    = (known after apply)
  + ebs_optimized              = (known after apply)
```

Terraform apply:

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

```
# aws_instance.web_server will be created
+ resource "aws_instance" "web_server" {
  + ami              = "ami-0e38835daf6b8a2b9"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + disable_api_stop  = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized     = (known after apply)
```

Ln 22, Col 2 Spaces: 4 UTF-8 CRLF {} Te

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.web_server: Creating...

aws_instance.web_server: Still creating... [00m10s elapsed]

aws_instance.web_server: Creation complete after 14s [id=i-039561e76780e9f57]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

PS C:\Users\DELL\Desktop\Terraform> █

Ln 22, C

verify:

Instances (1/1) [Info](#) Last updated 2 minutes ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets: Running

Instance state = running [Clear filters](#)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
Terraform-WebServer	i-039561e76780e9f57	Running	t3.micro	Initializing	ap-south-1a	ec2-13-233-138-51.ap-south-1.compute.amazonaws.com

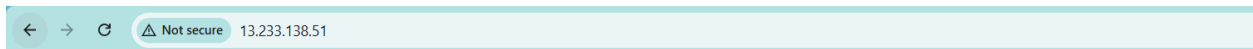
i-039561e76780e9f57 (Terraform-WebServer)

[Details](#) | [Status and alarms](#) | [Monitoring](#) | [Security](#) | [Networking](#) | [Storage](#) | [Tags](#)

▼ Instance summary [Info](#)

Instance ID i-039561e76780e9f57	Public IPv4 address 13.233.138.51 open address	Private IPv4 addresses 172.31.35.213
IPv6 address -	Instance state Running	Public DNS ec2-13-233-138-51.ap-south-1.compute.amazonaws.com open address

Verify in browser: public IP :80



Apache Web Server Installed using Terraform

4). Set up **S3 as backend** for task 3.

Create s3 bucket: write a script

```
main.tf x terraform.tfstate
main.tf > terraform > backend "s3" > abc bucket
5 resource "aws_instance" "web_server" {
10   user_data = <<-EOF
16       echo "<h1>Apache Web Server Installed using Terraform</h1>" > /var/www/html/index.html
17   EOF
18
19   tags = {
20     Name = "Terraform-WebServer"
21   }
22 }
23
24
25 resource "aws_s3_bucket" "terraform_state" {
26   bucket = "my-terraform-state-bucket-0117"
27
28   tags = {
29     Name = "Terraform State Bucket"
30   }
31 }
```

Terraform init & terraform apply

```
PS C:\Users\DELL\Desktop\Terraform> terraform apply
aws_instance.web_server: Refreshing state... [id=i-039561e76780e9f57]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.terraform_state will be created
+ resource "aws_s3_bucket" "terraform_state" {
+   acceleration_status      = (known after apply)
+   acl                     = (known after apply)
+   arn                    = (known after apply)
+   bucket                  = "my-terraform-state-bucket-0117"
+   bucket_domain_name      = (known after apply)
```

verify:

Buckets

General purpose buckets All AWS Regions Directory buckets

General purpose buckets (1) Info

Copy ARN Empty Delete Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
my-terraform-state-bucket-0117	Asia Pacific (Mumbai) ap-south-1	June 16, 2026, 15:35:35 (UTC+05:30)

Account snapshot Info

Updated daily

[View dashboard](#)

Storage Lens provides visibility into storage usage and activity trends.

External access summary Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

Write a script for store the state file in backend

```
main.tf  X  terraform.tfstate
main.tf > terraform > backend "s3" > abc bucket
22 }
23
24
25 resource "aws_s3_bucket" "terraform_state" {
26     bucket = "my-terraform-state-bucket-0117"
27
28     tags = {
29         Name = "Terraform State Bucket"
30     }
31 }
32
33 terraform {
34     backend "s3" {
35         bucket = "my-terraform-state-bucket-0117"
36         key    = "ec2-httpd/terraform.tfstate"
37         region = "ap-south-1"
38     }
39 }
```

Terraform init:

```
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...
Do you want to copy existing state to the new backend?
Pre-existing state was found while migrating the previous "local" backend to the
newly configured "s3" backend. No existing state was found in the newly
configured "s3" backend. Do you want to copy this state to the new "s3"
backend? Enter "yes" to copy and "no" to start with an empty state.

Enter a value: yes
```

Terraform apply:

```
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\DELL\Desktop\Terraform> terraform apply
aws_s3_bucket.terraform_state: Refreshing state... [id=my-terraform-state-bucket-0117]
aws_instance.web_server: Refreshing state... [id=i-039561e76780e9f57]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are n

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\DELL\Desktop\Terraform>
```

verify:

my-terraform-state-bucket-0117 [Info](#)

[Objects](#) [Metadata](#) [Properties](#) [Permissions](#) [Metrics](#) [Management](#) [File systems - new](#) [Access Points](#)

Objects (1) [Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	ec2-httpd/	Folder	-	-	-

ec2-httpd/ [Copy S3 URI](#)

[Objects](#) [Properties](#)

Objects (1) [Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	terraform.tfstate	tfstate	June 16, 2026, 15:38:04 (UTC+05:30)	8.4 KB	Standard

5). Set up **DynamoDB locking** for task 3.

Write a script for creating a dynamodb locking:

```
main.tf
31 }
32 }
33 }
34 resource "aws_dynamodb_table" "terraform_lock" {
35     name           = "terraform-lock"
36     billing_mode   = "PAY_PER_REQUEST"
37     hash_key       = "LockID"
38 }
39 attribute {
40     name = "LockID"
41     type = "s"
42 }
43 }
44 }
45 }

backend.tf
1 terraform {
2     backend "s3" {
3         bucket = "my-terraform-state-bucket-0117"
4         key     = "terraform.tfstate"
5         region  = "ap-south-1"
6         dynamodb_table = "terraform-lock"
7     }
8 }
9 }
```

verify:

Tables (1) [info](#)

Last updated
June 16, 2026, 16:22 (UTC+5:30)



Actions ▾

Delete

Create table ▾

Filter by tag
Any tag key ▾

Filter by tag value
Any tag value ▾

< 1 > ⚙

☐	Name ▲	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite ▾	Read capacity mode
☐	terraform-lock	✔ Active	LockID (S)	-	0	0	⊖ Off	☆	On-demand